Experiment 1: Linux Environment & File Management

```
# 1. Create and navigate to the practical directory
mkdir -p ~/OS_Lab/Exp1 && cd ~/OS_Lab/Exp1

# 2. Create sample files
touch file1.txt file2.txt file3.txt

# 3. View detailed file permissions and ownership
ls -la

# 4. Modify permissions (User: rwx, Group: r--, Others: r--)
chmod 744 file1.txt
ls -l file1.txt

# 5. Spawn a background process and observe PID
sleep 500 &

# 6. Monitor process status
ps -aux | grep sleep

# 7. Terminate the background process using its PID
kill -9 <PID_FROM_PREVIOUS_STEP>
```

Experiment 2: Shell Scripting and System Automation
Program 1: Control & Looping Menu Automation (practical2.sh)

```
#!/bin/bash
# Experiment 2: System Automation Shell Script

echo "========================================="
echo "   SYSTEM AUTOMATION & CONTROL SCRIPT   "
echo "========================================="

while true; do
    echo ""
    echo "1. Check Even / Odd Number"
    echo "2. Print Numbers 1 to N (Loop Demonstration)"
    echo "3. File Operations (Check Existence & Create)"
    echo "4. Calculate Sum using Function"
    echo "5. Exit"
    echo -n "Enter your choice [1-5]: "
    read choice

    case $choice in
        1)
            echo -n "Enter an integer: "
            read num
            if [ $((num % 2)) -eq 0 ]; then
                echo "Result: $num is EVEN."
            else
                echo "Result: $num is ODD."
            fi
```

```
        ;;
    2)
        echo -n "Enter N: "
        read n
        echo "Printing numbers 1 to $n:"
        for (( i=1; i<=n; i++ )); do
            echo -n "$i "
        done
        echo ""
        ;;
    3)
        echo -n "Enter filename to check/create: "
        read fname
        if [ -f "$fname" ]; then
            echo "File '$fname' exists. Contents:"
            cat "$fname"
        else
            echo "File '$fname' does not exist. Creating file..."
            echo "Automated Log Entry - $(date)" > "$fname"
            echo "File created successfully."
        fi
        ;;
    4)
        calculate_sum() {
            local a=$1
            local b=$2
            echo "Sum of $a and $b is: $((a + b))"
        }
        echo -n "Enter first number: "
        read num1
        echo -n "Enter second number: "
        read num2
        calculate_sum $num1 $num2
        ;;
    5)
        echo "Exiting script. Goodbye!"
        exit 0
        ;;
    *)
        echo "Invalid choice! Please select 1-5."
        ;;
  esac
done
```

Experiment 3: Process Creation, CPU Scheduling & Performance Analysis
Program 1: Process Creation using fork() (process.c)

```
#include <stdio.h>
#include <unistd.h>
```

```c
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    printf("[Main] Creating a child process using fork()...\n");
    pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        return 1;
    }
    else if (pid == 0) {
        // Child Process execution branch
        printf("[Child] Process created successfully.\n");
        printf("[Child] Child PID : %d\n", getpid());
        printf("[Child] Parent PID: %d\n", getppid());
    }
    else {
        // Parent Process execution branch
        wait(NULL); // Wait for child to complete execution
        printf("[Parent] Child execution completed.\n");
        printf("[Parent] Parent PID: %d\n", getpid());
        printf("[Parent] Child PID : %d\n", pid);
    }

    return 0;
}
```

Program 2: First-Come, First-Served (FCFS) CPU Scheduling (fcfs.c)

```c
#include <stdio.h>

int main() {
    int n, bt[20], wt[20], tat[20];
    float total_wt = 0, total_tat = 0;

    printf("Enter total number of processes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter Burst Time for Process P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // Waiting time for first process is 0
    wt[0] = 0;

    // Calculate waiting time for all remaining processes
    for (int i = 1; i < n; i++) {
```

```
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    // Calculate turnaround time
    for (int i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
        total_wt += wt[i];
        total_tat += tat[i];
    }

    printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");
    for (int i = 0; i < n; i++) {
        printf("P%d\t%d\t\t%d\t\t%d\n", i + 1, bt[i], wt[i], tat[i]);
    }

    printf("\nAverage Waiting Time    = %.2f\n", total_wt / n);
    printf("Average Turnaround Time = %.2f\n", total_tat / n);

    return 0;
}
```

Program 3: Non-Preemptive Shortest Job First (SJF) Scheduling (sjf.c)

```
#include <stdio.h>

int main() {
    int n, bt[20], p[20], wt[20], tat[20], temp;
    float total_wt = 0, total_tat = 0;

    printf("Enter total number of processes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        p[i] = i + 1;
        printf("Enter Burst Time for Process P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // Sort processes by Burst Time in ascending order
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (bt[i] > bt[j]) {
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;

                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
```

```
    }

    wt[0] = 0;
    for (int i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    for (int i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
        total_wt += wt[i];
        total_tat += tat[i];
    }

    printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");
    for (int i = 0; i < n; i++) {
        printf("P%d\t%d\t\t%d\t\t%d\n", p[i], bt[i], wt[i], tat[i]);
    }

    printf("\nAverage Waiting Time    = %.2f\n", total_wt / n);
    printf("Average Turnaround Time = %.2f\n", total_tat / n);

    return 0;
}
```

Program 4: Round Robin (RR) CPU Scheduling (rr.c)

```
#include <stdio.h>

int main() {
    int n, tq, time = 0, done;
    int bt[20], rem_bt[20], wt[20], tat[20];
    float total_wt = 0, total_tat = 0;

    printf("Enter total number of processes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter Burst Time for Process P%d: ", i + 1);
        scanf("%d", &bt[i]);
        rem_bt[i] = bt[i];
        wt[i] = 0;
    }

    printf("Enter Time Quantum: ");
    scanf("%d", &tq);

    do {
        done = 1;
        for (int i = 0; i < n; i++) {
            if (rem_bt[i] > 0) {
                done = 0;
```

```
            if (rem_bt[i] > tq) {
                time += tq;
                rem_bt[i] -= tq;
            } else {
                time += rem_bt[i];
                wt[i] = time - bt[i];
                rem_bt[i] = 0;
            }
        }
    }
} while (done == 0);

for (int i = 0; i < n; i++) {
    tat[i] = bt[i] + wt[i];
    total_wt += wt[i];
    total_tat += tat[i];
}

printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");
for (int i = 0; i < n; i++) {
    printf("P%d\t%d\t\t%d\t\t%d\n", i + 1, bt[i], wt[i], tat[i]);
}

printf("\nAverage Waiting Time    = %.2f\n", total_wt / n);
printf("Average Turnaround Time = %.2f\n", total_tat / n);

return 0;
}
```

Experiment 4: Multithreading & Resource Utilization
Program 1: Basic POSIX Thread Creation (thread.c)

```
#include <stdio.h>
#include <pthread.h>

void *thread_function(void *arg) {
    int id = *(int *)arg;
    printf("[Thread %d] Execution started. Thread ID = %lu\n", id, pthread_self());
    return NULL;
}

int main() {
    pthread_t t1, t2;
    int id1 = 1, id2 = 2;

    pthread_create(&t1, NULL, thread_function, &id1);
    pthread_create(&t2, NULL, thread_function, &id2);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
```

```
    printf("[Main] Both threads completed execution successfully.\n");
    return 0;
}
```

Program 2: Single-Threaded vs Multithreaded Execution Time Comparison (compare.c)

```
#include <stdio.h>
#include <pthread.h>
#include <time.h>

#define N 100000000L

void *cpu_work(void *arg) {
    volatile double sum = 0;
    for (long i = 0; i < N / 2; i++) {
        sum += i * 0.5;
    }
    return NULL;
}

int main() {
    clock_t start, end;
    pthread_t t1, t2;

    // --- Sequential (Single Thread) Execution ---
    start = clock();
    cpu_work(NULL);
    cpu_work(NULL);
    end = clock();
    double seq_time = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Sequential Execution Time   : %.4f seconds\n", seq_time);

    // --- Concurrent (Multithreaded) Execution ---
    start = clock();
    pthread_create(&t1, NULL, cpu_work, NULL);
    pthread_create(&t2, NULL, cpu_work, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    end = clock();
    double multi_time = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Multithreaded Execution Time: %.4f seconds\n", multi_time);

    return 0;
}
```

Experiment 5: Inter-Process Communication (IPC)
Program 1: One-Way IPC via Unnamed Pipe (pipe_ipc.c)

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
```

```c
int main() {
    int pipe_fd[2];
    char write_msg[] = "Hello from Parent Process via Pipe!";
    char read_buffer[100];

    if (pipe(pipe_fd) == -1) {
        perror("Pipe creation failed");
        return 1;
    }

    if (fork() == 0) {
        // Child Process: Close write end, read message
        close(pipe_fd[1]);
        read(pipe_fd[0], read_buffer, sizeof(read_buffer));
        printf("[Child] Message Received: %s\n", read_buffer);
        close(pipe_fd[0]);
    } else {
        // Parent Process: Close read end, write message
        close(pipe_fd[0]);
        write(pipe_fd[1], write_msg, strlen(write_msg) + 1);
        printf("[Parent] Message Sent.\n");
        close(pipe_fd[1]);
    }

    return 0;
}
```

Program 2: Shared Memory IPC (shm_write.c)

```c
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <string.h>

int main() {
    // Generate unique key and access shared memory segment
    key_t key = ftok("shmfile", 65);
    int shmid = shmget(key, 1024, 0666 | IPC_CREAT);

    // Attach shared memory segment
    char *str = (char*) shmat(shmid, (void*)0, 0);

    strcpy(str, "Shared Memory IPC Active - Data Written!");
    printf("Data Written to Shared Memory: %s\n", str);

    // Detach from shared memory segment
    shmdt(str);

    return 0;
}
```

Program 3: System V Message Queue IPC (msgqueue.c)

```c
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <string.h>

struct msg_buffer {
    long msg_type;
    char msg_text[100];
};

int main() {
    key_t key = ftok("msgqfile", 65);
    int msgid = msgget(key, 0666 | IPC_CREAT);

    struct msg_buffer message;
    message.msg_type = 1;
    strcpy(message.msg_text, "System V Message Queue Payload Delivered!");

    msgsnd(msgid, &message, sizeof(message), 0);
    printf("Message queued successfully: %s\n", message.msg_text);

    return 0;
}
```

Experiment 6: Synchronization Mechanisms for Concurrent Systems
Program 1: Critical Section Protection using Mutex (mutex.c)

```c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock;
int shared_counter = 0;

void *critical_section(void *arg) {
    int id = *(int *)arg;

    pthread_mutex_lock(&lock);
    printf("[Thread %d] Entered Critical Section.\n", id);
    shared_counter++;
    printf("[Thread %d] Shared Counter = %d\n", id, shared_counter);
    sleep(1);
    printf("[Thread %d] Exiting Critical Section.\n", id);
    pthread_mutex_unlock(&lock);

    return NULL;
}

int main() {
    pthread_t t1, t2;
```

```
    int id1 = 1, id2 = 2;

    pthread_mutex_init(&lock, NULL);

    pthread_create(&t1, NULL, critical_section, &id1);
    pthread_create(&t2, NULL, critical_section, &id2);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&lock);
    return 0;
}
```

Program 2: Producer-Consumer Problem using Semaphores (prodcons.c)

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define TOTAL_ITEMS 10

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty_slots;
sem_t full_slots;
pthread_mutex_t mutex;

void *producer(void *arg) {
    for (int i = 1; i <= TOTAL_ITEMS; i++) {
        sem_wait(&empty_slots);
        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        printf("[Producer] Produced Item %d at Slot %d\n", i, in);
        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full_slots);
        usleep(100000);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 1; i <= TOTAL_ITEMS; i++) {
        sem_wait(&full_slots);
        pthread_mutex_lock(&mutex);
```

```
        int item = buffer[out];
        printf("  [Consumer] Consumed Item %d from Slot %d\n", item, out);
        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty_slots);
        usleep(150000);
    }
    return NULL;
}

int main() {
    pthread_t prod_thread, cons_thread;

    sem_init(&empty_slots, 0, BUFFER_SIZE);
    sem_init(&full_slots, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&prod_thread, NULL, producer, NULL);
    pthread_create(&cons_thread, NULL, consumer, NULL);

    pthread_join(prod_thread, NULL);
    pthread_join(cons_thread, NULL);

    sem_destroy(&empty_slots);
    sem_destroy(&full_slots);
    pthread_mutex_destroy(&mutex);

    return 0;
}
```

Experiment 7: Deadlock Detection, Avoidance & Recovery
Program 1: Banker's Algorithm for Deadlock Avoidance (banker.c)

```
#include <stdio.h>
#include <stdbool.h>

#define P 5
#define R 3

void calculateNeed(int need[P][R], int max[P][R], int alloc[P][R]) {
    for (int i = 0; i < P; i++)
        for (int j = 0; j < R; j++)
            need[i][j] = max[i][j] - alloc[i][j];
}

bool isSafeState(int processes[], int avail[], int max[P][R], int alloc[P][R]) {
    int need[P][R];
    calculateNeed(need, max, alloc);
```

```
    bool finish[P] = {false};
    int safeSeq[P];
    int work[R];

    for (int i = 0; i < R; i++) work[i] = avail[i];

    int count = 0;
    while (count < P) {
        bool found = false;
        for (int p = 0; p < P; p++) {
            if (!finish[p]) {
                int j;
                for (j = 0; j < R; j++) {
                    if (need[p][j] > work[j]) break;
                }
                if (j == R) {
                    for (int k = 0; k < R; k++) work[k] += alloc[p][k];
                    safeSeq[count++] = p;
                    finish[p] = true;
                    found = true;
                }
            }
        }
        if (!found) {
            printf("\nSystem is NOT in a safe state!\n");
            return false;
        }
    }

    printf("\nSystem is in a SAFE state.\nSafe Sequence: ");
    for (int i = 0; i < P; i++) {
        printf("P%d ", safeSeq[i]);
    }
    printf("\n");
    return true;
}

int main() {
    int processes[] = {0, 1, 2, 3, 4};
    int avail[] = {3, 3, 2};

    int max[P][R] = {
        {7, 5, 3},
        {3, 2, 2},
        {9, 0, 2},
        {2, 2, 2},
        {4, 3, 3}
    };

    int alloc[P][R] = {
        {0, 1, 0},
```

```
        {2, 0, 0},
        {3, 0, 2},
        {2, 1, 1},
        {0, 0, 2}
    };

    isSafeState(processes, avail, max, alloc);
    return 0;
}
```

Program 2: Deadlock Detection & Recovery Simulation (detect.c)

```
#include <stdio.h>
#include <stdbool.h>

#define P 3
#define R 3

int main() {
    int avail[] = {0, 0, 0};
    int alloc[P][R] = {{0,1,0}, {0,0,1}, {1,0,0}};
    int request[P][R] = {{0,0,1}, {1,0,0}, {0,1,0}};
    int work[R];

    for (int i = 0; i < R; i++) work[i] = avail[i];
    bool finish[P] = {false};

    printf("Executing Deadlock Detection Analysis...\n");
    bool deadlocked = false;
    for (int i = 0; i < P; i++) {
        if (!finish[i]) {
            printf("Deadlock detected! Process P%d is deadlocked in circular wait.\n", i);
            deadlocked = true;
        }
    }

    if (deadlocked) {
        printf("\n[Recovery Mechanism Triggered]: Aborting Process P2 to break circular wait chain...\n");
        for (int m = 0; m < R; m++) work[m] += alloc[2][m];
        finish[2] = true;
        printf("Process P2 aborted. Resources reclaimed successfully.\n");
    }

    return 0;
}
```

Experiment 8: Memory Management & Page Replacement Strategies
Program 1: Address Translation Simulation for Paging (paging.c)

```
#include <stdio.h>
```

```c
#define PAGE_SIZE 1024

int main() {
    int pageTable[10], numPages, logicalAddr, page, offset, physicalAddr;

    printf("Enter number of pages: ");
    scanf("%d", &numPages);

    for (int i = 0; i < numPages; i++) {
        printf("Enter physical frame number for page %d: ", i);
        scanf("%d", &pageTable[i]);
    }

    printf("Enter Logical Address: ");
    scanf("%d", &logicalAddr);

    page = logicalAddr / PAGE_SIZE;
    offset = logicalAddr % PAGE_SIZE;

    if (page >= numPages) {
        printf("Invalid Page Reference! (Page Fault - Out of logical bounds)\n");
    } else {
        physicalAddr = pageTable[page] * PAGE_SIZE + offset;
        printf("\n--- Address Translation Result ---\n");
        printf("Page Number      = %d\n", page);
        printf("Offset           = %d\n", offset);
        printf("Physical Address = %d\n", physicalAddr);
    }

    return 0;
}
```

Program 2: FIFO Page Replacement Algorithm (fifo.c)

```c
#include <stdio.h>

int main() {
    int pages[20], n, frames[10], frameCount, faults = 0, front = 0;

    printf("Enter number of pages in reference string: ");
    scanf("%d", &n);

    printf("Enter reference string: ");
    for (int i = 0; i < n; i++) scanf("%d", &pages[i]);

    printf("Enter number of frames: ");
    scanf("%d", &frameCount);

    for (int i = 0; i < frameCount; i++) frames[i] = -1;

    for (int i = 0; i < n; i++) {
```

```
        int found = 0;
        for (int j = 0; j < frameCount; j++) {
            if (frames[j] == pages[i]) {
                found = 1;
                break;
            }
        }
        if (!found) {
            frames[front] = pages[i];
            front = (front + 1) % frameCount;
            faults++;
        }
    }

    printf("\nTotal Page Faults (FIFO) = %d\n", faults);
    printf("Hit Ratio                = %.2f\n", (float)(n - faults) / n);

    return 0;
}
```

Program 3: LRU Page Replacement Algorithm (lru.c)

```
#include <stdio.h>

int main() {
    int pages[20], n, frames[10], recent[10], frameCount, faults = 0;

    printf("Enter number of pages in reference string: ");
    scanf("%d", &n);

    printf("Enter reference string: ");
    for (int i = 0; i < n; i++) scanf("%d", &pages[i]);

    printf("Enter number of frames: ");
    scanf("%d", &frameCount);

    for (int i = 0; i < frameCount; i++) frames[i] = -1;

    for (int i = 0; i < n; i++) {
        int found = -1;
        for (int j = 0; j < frameCount; j++) {
            if (frames[j] == pages[i]) {
                found = j;
                break;
            }
        }

        if (found == -1) {
            int lruIndex = 0, minRecent = recent[0];
            for (int j = 0; j < frameCount; j++) {
                if (frames[j] == -1) {
```

```
                lruIndex = j;
                break;
            } else if (recent[j] < minRecent) {
                minRecent = recent[j];
                lruIndex = j;
            }
        }
        frames[lruIndex] = pages[i];
        recent[lruIndex] = i;
        faults++;
    } else {
        recent[found] = i;
    }
}

printf("\nTotal Page Faults (LRU)  = %d\n", faults);
printf("Hit Ratio                = %.2f\n", (float)(n - faults) / n);

return 0;
}
```

Experiment 9: File System & Disk Scheduling Analysis
Program 1: FCFS Disk Scheduling Algorithm (fcfs_disk.c)

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n, head, seek_total = 0, req[50];

    printf("Enter total number of requests: ");
    scanf("%d", &n);

    printf("Enter request sequence: ");
    for (int i = 0; i < n; i++) scanf("%d", &req[i]);

    printf("Enter initial read/write head position: ");
    scanf("%d", &head);

    for (int i = 0; i < n; i++) {
        seek_total += abs(req[i] - head);
        head = req[i];
    }

    printf("\nTotal Head Movement (FCFS) = %d tracks\n", seek_total);
    printf("Average Seek Time          = %.2f tracks\n", (float)seek_total / n);

    return 0;
}
```

Program 2: SSTF Disk Scheduling Algorithm (sstf_disk.c)

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n, head, seek_total = 0, req[50], visited[50] = {0};

    printf("Enter total number of requests: ");
    scanf("%d", &n);

    printf("Enter request sequence: ");
    for (int i = 0; i < n; i++) scanf("%d", &req[i]);

    printf("Enter initial read/write head position: ");
    scanf("%d", &head);

    for (int i = 0; i < n; i++) {
        int min_dist = 99999, min_idx = -1;
        for (int j = 0; j < n; j++) {
            if (!visited[j] && abs(req[j] - head) < min_dist) {
                min_dist = abs(req[j] - head);
                min_idx = j;
            }
        }
        seek_total += min_dist;
        head = req[min_idx];
        visited[min_idx] = 1;
    }

    printf("\nTotal Head Movement (SSTF) = %d tracks\n", seek_total);
    printf("Average Seek Time          = %.2f tracks\n", (float)seek_total / n);

    return 0;
}
```

Experiment 10: Containerization & Resource Monitoring for AI/ML Workloads
File 1: Python AI/ML Application (app.py)

```python
import time
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

def run_pipeline():
    print("[Docker Container] Loading Iris Dataset...")
    data = load_iris()
    X_train, X_test, y_train, y_test = train_test_split(
        data.data, data.target, test_size=0.2, random_state=42
    )
```

```
    print("[Docker Container] Training Random Forest Model...")
    model = RandomForestClassifier(n_estimators=100)
    model.fit(X_train, y_train)

    predictions = model.predict(X_test)
    accuracy = accuracy_score(y_test, predictions)
    print(f"[Docker Container] Model Accuracy: {accuracy * 100:.2f}%")

    print("[Docker Container] Process active for resource monitoring...")
    time.sleep(30)

if __name__ == "__main__":
    run_pipeline()
```

File 2: Containerization Setup Specification (Dockerfile)

```
# Use explicit python slim image for lightweight deployment
FROM python:3.10-slim

WORKDIR /app

# Install ML dependencies
RUN pip install --no-cache-dir scikit-learn

COPY app.py .

CMD ["python", "app.py"]
```

Execution Commands (Terminal Workflow)

```
# 1. Build the Docker Image
docker build -t aiml-workload:1.0 .

# 2. Run container in background
docker run -d --name aiml-app-container aiml-workload:1.0

# 3. Monitor container resource metrics in real-time
docker stats aiml-app-container
```